

Programação Orientada a Objetos

Capítulo 6 — Linguagem de Programação C#

Aula 06

O que veremos hoje?

- 1 C#: origem, *.NET Framework* e *Visual Studio*
- 2 Criação de classes: `namespace`, `using`, `Main`
- 3 Métodos em C#: `void`, retorno, *getters/setters*
- 4 Modificadores de acesso (encapsulamento)
- 5 Construtores múltiplos (polimorfismo)
- 6 **Herança com ':' + Comparativo *Java vs. C#***



6.1 Linguagem C#

Microsoft, .NET e alta produtividade

Origem e características do C#

- Pronuncia-se '*C sharp*' — desenvolvida pela *Microsoft* em 2000.
- Criada para utilização do conceito *.NET: frameworks* com funções pré-implementadas.
- Foco em OO e agilidade na produção do resultado final — fácil aprendizagem e alta produtividade.
- Todas as bibliotecas do *.NET Framework* foram escritas em C# pelo alto desempenho.
- Sintaxe baseada em C — familiar para quem já conhece *Java*, *C++* ou *C*.
- *IDE* principal: *Visual Studio (Microsoft)* — integração nativa e completa.

Estrutura básica de um projeto C#

- Ao criar um projeto no *Visual Studio*, a classe principal **Program.cs** é gerada com o método **Main**.
- **Main** é o ponto de entrada — primeira coisa executada ao iniciar o programa.
- **'namespace'**: organiza classes em pacotes lógicos (equivalente ao **'package'** do *Java*).
- **'using'**: declara bibliotecas utilizadas pela classe (ex.: `using System;` para `Console.WriteLine`).
- Nova classe: botão direito no pacote → Adicionar → Novo Item → Classe.
- `Console.WriteLine("texto")` = equivalente ao `System.out.println("texto")` do *Java*.

Hello World em C#

```
using System;

namespace ConsoleApp1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

using = importar biblioteca | *namespace* = pacote | *Main* = ponto de entrada

Dinâmica 1 — De Java para C#: Traduzindo

- Em duplas, 'traduzam' o seguinte código *Java* para *C#*:

- Java: package item1;
 - ```
public class Carro {
 public String modelo;
 public void ligar() {
 System.out.println("Ligou!");
 }
}
```

- Dicas:
  - `package` → `namespace`;
  - `System.out.println` → `Console.WriteLine`;
  - `import` → `using`

- Identifiquem as semelhanças e diferenças de sintaxe.
- Tempo: 5 minutos.

# Resposta — Dinâmica 1

- *C#*:
  - `using System;`  
`namespace Item1 {`  
 `class Carro {`  
 `public string modelo;`  
 `public void ligar() {`  
 `Console.WriteLine("Ligou!");`  
 `}`  
 `}`  
`}`

- Semelhanças:
  - `{ }` para escopo,
  - `public/void`,
  - nomes de classe com maiúscula,
  - ponto-e-vírgula.
- Diferenças:
  - `package` → `namespace`,
  - `import` → `using`,
  - `System.out.println` → `Console.WriteLine`,
  - `String` → `string` (minúsculo).

A migração entre *Java* e *C#* é muito natural graças à base sintática comum do *C*.





## 6.2 Métodos em C#

*void, retorno e o padrão getter/setter*

# Criando métodos em C#

- Métodos void: sem retorno — executam ações (ex.: `public void ligar() { ... }`).
- Métodos com retorno: declaram tipo antes do nome + **return** (ex.: `public int calcularAno(int ano) { ... }`).
- Sintaxe muito semelhante ao *Java* — facilita migração entre as linguagens.
- Métodos trocam mensagens entre objetos, cada um realizando sua tarefa específica.
- Boas práticas: nomes de métodos descritivos, com verbos (ligar, calcular, exibir).

# Classe Carro em C# com métodos

```
class Carro
{
 public float motor;
 public string modelo;
 public int ano;

 public void ligar() // Sem retorno
 {
 Console.WriteLine("Carro Ligou!");
 }

 public int calcularAno(int ano) // Com retorno
 {
 return ano - this.ano;
 }
}
```

*Praticamente idêntico ao Java — a diferença é `Console.WriteLine` vs `System.out.println`*

## Dinâmica 2 — Criando Classe Completa em C#

- Individualmente, crie a classe 'Produto' em C# com:
- Atributos: `private string nome;` `private float preco;` `private int estoque;`
- *Getters* e *Setters* para cada atributo (`public string getNome()` / `public void setNome(string nome)`).
- Método `void`: `exibirInfo()` → imprime nome, preço e estoque.
- Método com retorno: `calcularValorTotal()` → retorna `preco * estoque`.
- Tempo: 7 minutos.

## Exemplo de resposta — Dinâmica 2

```
class Produto {
 private string nome;
 private float preco;
 private int estoque;

 public string getNome() {
 return nome;
 }

 public void setNome(string nome) {
 this.nome = nome;
 }
}
```

```
 public float getPreco() {
 return preco;
 }

 public void setPreco(float preco) {
 this.preco = preco;
 }

 public void exibirInfo() {
 Console.WriteLine(nome + " - R$" + preco + " - Estoque: " + estoque);
 }

 public float calcularValorTotal() {
 return this.preco * this.estoque;
 }
}
```

*Getters/setters* públicos controlam o acesso aos atributos privados = encapsulamento!



## 6.3 Modificadores de Acesso

*Encapsulamento em C#: private, public e getters/setters*

# Encapsulamento com modificadores em C#

- Mesmo conceito do *Java*: **public**, **private**, **protected** e *internal* (equivalente ao default).
- Atributos **private** + *getters/setters* **public** = padrão de encapsulamento em C#.
- **get**: permite ler o valor de um atributo privado de fora da classe.
- **set**: permite gravar um valor num atributo privado, controlando a entrada de dados.
- Resultado: outras classes acessam apenas o necessário, detalhes internos ficam protegidos.
- C# também oferece Properties (**get**; **set**;) como atalho sintático.

# Classe Pessoa com encapsulamento em C#

```
class Pessoa
{
 private string nome;
 private int idade;
 private long cpf;

 public void setNome(string nome) { this.nome = nome; }
 public string getNome() { return nome; }

 public void setIdade(int idade) { this.idade = idade; }
 public int getIdade() { return idade; }

 public void exibirDados() {
 Console.WriteLine(nome + " - " + idade + " - " + cpf);
 }
}
```

Atributos **private** + métodos **get/set public** = encapsulamento clássico



## Dinâmica 3 — Público ou Privado em C#?

- Em duplas, para a classe 'ContaCorrente', classifiquem cada membro:
- Atributos: `saldo`, `titular`, `agencia`, `senha`, `limiteEspecial`.
- Métodos: `depositar()`, `sacar()`, `getSaldo()`, `setSenha()`, `transferir()`, `validarSenha()`.
- Para cada um: defina como **public** ou **private** e justifique.
- Como o titular de outra conta consegue verificar saldo se ele é **private**?  
(dica: *getter*)
- Tempo: 5 minutos.

# Respostas — Dinâmica 3

- **private:** `saldo`, `senha`, `limiteEspecial`, `validarSenha()` — detalhes internos sensíveis;
- **public:** `depositar()`, `sacar()`, `getSaldo()`, `transferir()`, `titular` (leitura via *getter*), `agencia` (leitura via *getter*);
- `setSenha()`: pode ser **public** mas com validação interna (exigir senha antiga antes de trocar);
- O `saldo` é lido por outras classes através de `getSaldo()` — nunca diretamente;
- Princípio: exponha o mínimo necessário, proteja o máximo possível.



## 6.4 Construtores Múltiplos

*Polimorfismo em C#: sobrecarga de construtores*

# Construtores múltiplos em C#

- Polimorfismo via sobrecarga: vários métodos com mesmo nome, parâmetros diferentes;
- Construtor: tem o mesmo nome da classe — inicializa o objeto ao instanciar com `'new'`;
- Diferenciação: feita pela quantidade e tipos de parâmetros passados;
- Pode-se ter: construtor sem parâmetros, com dados parciais ou com todos os dados;
- Sintaxe idêntica ao *Java* — o conceito é universal em linguagens OO.

# Classe Aviao em C# com 3 construtores

```
class Aviao
{
 private string empresa;
 private string modelo;

 public Aviao() // Sem parâmetros
 { Console.WriteLine("E vdou!"); }

 public Aviao(int x, int y) // 2 inteiros
 { Console.WriteLine("Resultado = " + (x + y)); }

 public Aviao(string empresa, string modelo) // 2 strings
 { this.empresa = empresa; this.modelo = modelo; }
}
```

*3 construtores, mesmo nome 'Aviao', parâmetros diferentes = polimorfismo*

## Dinâmica 4 — Construtores em C#

- Individualmente, crie a classe 'Veículo' em C# com 3 construtores:
- Construtor 1: sem parâmetros → `Console.WriteLine("Veículo padrão criado")`.
- Construtor 2: recebe (`string marca`) → inicializa apenas `marca`.
- Construtor 3: recebe (`string marca, int ano, float preco`) → inicializa tudo.
- Crie também uma classe 'Loja' com `Main` que instancie 3 objetos `Veículo`, um por construtor.
- Tempo: 6 minutos.

# Exemplo de resposta — Dinâmica 4

```
public Veiculo() {
 Console.WriteLine("Veículo padrão criado");
}

public Veiculo(string marca) {
 this.marca = marca;
}

public Veiculo(string marca, int ano, float preco) {
 this.marca = marca; this.ano = ano;
 this.preco = preco;
}
```

- Na Loja:
  - `Veiculo v1 = new Veiculo();`
  - `Veiculo v2 = new Veiculo("Fiat");`
  - `Veiculo v3 = new Veiculo("Honda", 2024, 95000f);`
- Cada `'new Veiculo(...)'` aciona um construtor diferente — polimorfismo via sobrecarga.



## 6.5 Herança em C#

*Subclasses com ':' — reutilização de código*



# Herança em C#: dois-pontos ':'

- Em C#, herança é declarada com ':' (dois-pontos): `class Onca : Animal { };`
- Equivalente ao 'extends' do Java — mesmo conceito, sintaxe levemente diferente;
- Classe filha herda todos os atributos e métodos da classe pai;
- Classe filha pode adicionar atributos e métodos próprios, mais específicos;
- Exemplo: `Animal (especie, tamanho, cor)` → `Onca` herda tudo + `peso`, `velocidade`, `habitat`;
- Regra: uma onça É um animal, mas nem todo animal é uma onça.

# Exemplo: Animal → Onca em C#

```
class Animal // Superclasse
{
 private string especie;
 private float tamanho;
 private string cor;
 // getters e setters...
}

class Onca : Animal // Subclasse (herda de Animal)
{
 private float peso;
 private int velocidade;
 private string habitat;
 // getters e setters próprios...
}
```

Java: *extends* | C#: ':' — mesmo conceito, sintaxe diferente

## Dinâmica 5 — Herança Completa em C#

- Em duplas, criem uma hierarquia em *C#* para um sistema de e-commerce:
  - Superclasse 'Produto':
    - `private string nome, float preco + getters/setters + exibirInfo();`
  - Subclasse 'Eletronico : Produto': `private int voltagem, int garantiaMeses + getters/setters;`
  - Subclasse 'Roupa : Produto': `private string tamanho, string cor + getters/setters;`
  - Classe Main: instancie 1 Eletronico e 1 Roupa, preencha atributos e chame `exibirInfo()`.
- Tempo: 8 minutos.

# Exemplo de resposta — Dinâmica 5

```
class Produto {
 private string nome;
 private float preco;

 public void exibirInfo() {
 Console.WriteLine(nome + " R$" +
 preco);
 }

 /* getters/setters */
}

class Eletronico : Produto {
 private int voltagem;
 private int garantiaMeses;

 /* getters/setters próprios */
}
```

```
class Roupa : Produto {
 private string tamanho;
 private string cor;
 /* getters/setters próprios */
}
```

Main:

```
Eletronico tv = new Eletronico(); tv.setNome("TV");
tv.setPreco(2500f); tv.exibirInfo();
```

- Eletronico e Roupa herdam nome e preço de Produto — sem reescrita. Herança + encapsulamento juntos!

# Java vs. C# — Comparativo de Sintaxe

## Java

- `package` → organiza classes;
- `import` → importa bibliotecas;
- `extends` → herança;
- `System.out.println()`;
- `String` (maiúsculo);
- *JVM* (máquina virtual);
- *IDE: NetBeans / Eclipse.*

## C#

- `namespace` → organiza classes;
- `using` → importa bibliotecas;
- `:` (dois-pontos) → herança;
- `Console.WriteLine()`;
- `string` (minúsculo);
- *.NET Framework / CLR;*
- *IDE: Visual Studio;*

*"C# e Java compartilham a mesma base sintática do C e os mesmos 4 princípios OO — a migração entre elas é natural e produtiva."*

## Pontos-chave da aula

- C#: criada pela *Microsoft* (2000) para *.NET* — fácil aprendizagem, alta produtividade.
- `namespace = package` | `using = import` | `Console.WriteLine = System.out.println`.
- Encapsulamento: `private` + *getters/setters* públicos — mesmo padrão do *Java*.
- Polimorfismo: construtores múltiplos com parâmetros diferentes.
- Herança: `:` no C# equivale a `extends` no *Java* — mesma lógica, sintaxe levemente diferente.